

```

1(a)(i) public class Information {

    private String address;
    private String phone;
    private String email;

    public Information(String address, String phone, String
                        email) {

        this.address = address;
        this.phone = phone;
        this.email = email;

    }

    public String getAddress() { return address; }

    public String getPhone() { return phone; }

    public String getEmail() { return email; }

}

```

```

(ii) public record Information(String address, String phone, String email)
    {}

```

```

Information:: address()

```

(b) (ii)

```
public abstract class Contact {
```

```
    private String name;  
    private Information info;
```

```
    public Contact(String name, Information info) {  
        this.name = name;  
        this.info = info;  
    }
```

```
    public String getName() {  
        return name;  
    }
```

```
    public Information getInfo() {  
        return info;  
    }
```

```
}
```

(c)

```
public class BusinessContact extends Contact {
```

```
    private int accountNo;
```

```
    public BusinessContact(String name, Information info, int accountNo) {  
        super(name, info);
```

```
        this.accountNo = accountNo;  
    }
```

```
    public int getAccountNo() { return accountNo; }  
}
```

```
(d) public class ContactManager() {
```

```
    private ArrayList<Contact> contacts ;
```

```
    public ContactManager() {  
        contacts = new ArrayList<>();  
    }
```

```
    public void addPersonalContact(String name, Information info, String birthday){
```

```
        PersonalContact p = new PersonalContact(name, info, birthday);  
        contacts.add(p);  
    }
```

```
    ...
```

```
    public Contact find(String name) {  
        for (Contact c : contacts) {  
            if (c.getName().equals(name)) {  
                return c;  
            }  
        }  
    }
```

```
        return null;  
    }
```

JUnit

```
public class InformationTest {
```

```
    @Test
```

```
        public void addressShouldBeRetrievedCorrectly() {  
            Information info = new Information("address", ...);
```

```
            assertEquals("address", info.getAddress());
```

```
        }
```

```
    @Test
```

```
    :
```

```
}
```